

RATURAT Grégoire

Développeur Informatique

MODAAL - 25 Rue Saint-Michel, 69007 Lyon



Résumé du mémoire d'alternance professionnel

Master Expert en Système d'Information 2021-2023

Tuteur : MARTY Fabien

Manager – Ingénieur d'études et développement SIG

MODAAL 25 Rue Saint-Michel, 69007 Lyon

Problématique

Comment le développement de compétences en informatique peut-il contribuer à améliorer l'activité d'un bureau d'études en urbanisme.

Présentation

Qui suis-je

Je suis Grégoire Raturat, et je vais vous présenter mon parcours académique et professionnel, ainsi que la manière dont j'ai trouvé mon entreprise et mon école. Je suis passionné par l'informatique et j'ai toujours cherché à me développer dans ce domaine.

Je suis sortie du lycée **C. et P. Virlogeux** avec un baccalauréat scientifique spécialité science de la vie et de la terre (catégorie S), avec comme option Informatique et Science du Numérique (ISN). J'avais déjà mis un pied dans l'informatique personnellement et cette option m'a permis de confirmer mon intérêt pour le développement informatique.

Je me suis donc dirigé vers une licence Informatique à l'**Université Clermont Auvergne** (UCA) qui m'a permis d'approfondir mes connaissances et découvrir une autre facette du développement informatique. Arrivé en troisième année, je m'étais lassé des méthodes éducatives proposées par mon université, j'ai voulu tenter de passer mon année en alternance. Après de multiples démarches et propositions à mon Université, cela s'est soldé par un refus, bien mal accepté de mon côté après avoir rempli toutes les cases qu'il m'avait demandées.

Je suis donc parti à la recherche d'une école proposant l'alternance pour finir mon diplôme. Lyon devenait la destination qui me semblait la plus optimale pour trouver cette école. Après quelques recherches mon choix s'est arrêté sur deux écoles que j'ai contactées, j'ai finalement choisi **ISITECH** qui semblait remplir tous mes critères.

Suivant le cours des choses, il était temps de trouver une entreprise avec laquelle je pourrais travailler pour passer mon diplôme. L'école m'a permis de passer quelques entretiens avec des entreprises partenaires, et j'en ai trouvé quelques-unes de mon côté. Je cherchais principalement des entreprises proposant des missions orientées vers le développement informatique plutôt que le réseau.

J'ai finalement été rappelé par l'entreprise avec laquelle je travaille depuis maintenant 3 ans et avec laquelle je compte continuer, **MODAAL**.

Poste



Alternance informatique et développement logiciel – H / F

Tu souhaites évoluer au sein d'une petite équipe, passionnée par le web, la cartographie et les nouvelles technologies. A ton arrivée, tu découvriras nos différents métiers et plongeras dans le domaine de la web cartographie. Ton objectif sera de mettre en place des solutions techniques répondant aux besoins des projets de nos clients et d'outils internes.

Plus concrètement, tes missions seront de :

- Administrer et créer des bases de données (PostgreSQL)
- Développer des applications web (Node.js, Vue.js, OpenLayers, ...)
- Réaliser du scripting en Python
- Effectuer les tests et recettes des développements
- Administrer des serveurs Linux

Tes missions seront variées et évolueront dans le temps en fonction de tes motivations et intérêts.

De quoi as-tu besoin pour nous rejoindre ?

En formation supérieure, école d'ingénieur informatique ou Master d'université, tu as des notions en développement web Node.js, PHP, Java, Javascript, CSS, bases de données, ...

Tu disposes des bases sur les outils de développement (GIT) et une certaine appétence pour Linux !

Côté personnalité, tu es curieux et consciencieux dans ton travail. Tu aimes travailler en équipe tout en appréciant l'autonomie qu'on peut te laisser sur tes missions (nous ne serons pas derrière toi à chaque instant mais plutôt dès que tu en manifesteras le besoin 😊).

Comment déposer ta candidature ?

Tu souhaites travailler sur des projets concrets, réels et sérieux et mettre en application tes connaissances ?

L'équipe t'attend pour une alternance, en contrat d'apprentissage.

Envoie ton CV, les raisons qui te motivent et éventuellement tes réalisations (GitHub/GitLab) :

Edouard MOUGIN
Manager - Associé
06 21 00 55 17
edouard.mougin@modaal.fr

MODAAL, une société lyonnaise de 7 personnes

Notre entreprise est spécialisée en **analyse et accompagnement de projets d'aménagement du territoire**.

Nos méthodes et outils s'appuient sur la mobilisation de compétences métier en lien avec l'exploitation des dernières technologies web et de l'open data ! Nous développons **une approche innovante basée sur le croisement de données pluridisciplinaires et la création d'applications ciblées**.

Nos clients : Acteurs publics (collectivités territoriales, établissements publics fonciers) et d'entreprises (aménageurs, promoteurs, industriels)

Présentation

Bureau d'études en urbanisme fondé en 2014 par **Yann Milton**, situé dans Le 7^{ème} arrondissement de Lyon. Il est spécialisé dans le secteur du **conseil en architecture et en urbanisme**, plus précisément dans la **valorisation foncière** et l'accompagnement de **projets immobiliers** ou **d'aménagements**.

« Spécialiste du « foncier et de ce que l'on en fait », MODAAL est en mesure de vous proposer un panel de solutions et expertises dédiées à la valorisation foncière et au développement territorial. Trait d'union entre les entreprises et les collectivités territoriales, MODAAL connaît très bien les enjeux respectifs des acteurs publics (urbanisme, développement) et privés (recherche d'efficacité). » [Linkedin](#)



Localisation société/ fond de plan : OpenStreetMap

MODAAL intervient sur la France entière au service des **acteurs publics** (collectivités territoriales, EPF...) et des **opérateurs privés** (foncières, promoteurs, aménageurs, industriels) pour les accompagner sur les dimensions stratégiques, économiques et pré-opérationnelles de leurs projets.

MODAAL possède aussi une équipe en charge du développement des **outils digitaux** et des prestations associées. Principalement tourné vers une approche innovante avec le **croisement de données pluridisciplinaires**, l'utilisation des dernières **technologies web** et de **données ouvertes**.

Domaines d'intervention

- **Gestion foncière territoriale.**

Accompagnement des territoires vers une gestion maîtrisée et responsable de la ressource foncière, grâce à une approche globale, qui intègre les différents usages du foncier (habitat, activité, commerce, équipements...) et les différentes échelles d'intervention (du SCOT à la déclinaison locale). Avec des propositions de gamme complète de prestations allant de l'étude du parcellaire ou de la dureté foncière, jusqu'à la définition et l'évaluation des politiques foncières.

- **Offre d'accueil et attractivité économique.**

Services d'aides pour les territoires à planifier, optimiser et valoriser leurs offres d'accueil à destination des entreprises. Réalisations des études de marché, diagnostics territoriaux, stratégies, schémas et plans d'actions relatifs au foncier économique, à l'immobilier d'entreprise et plus globalement à l'attractivité.

- **Transformation de sites.**

Interventions sur des projets d'aménagement, de requalification ou de réhabilitation à l'échelle d'un site, pour aider les maîtrises d'ouvrage à définir, positionner et surtout sécuriser leurs projets. Réalisations des études d'opportunité, de positionnement, de programmation, de préféabilité et de montage, ainsi qu'un appui au pilotage du projet (AMO, suivi...).

- **Valorisation foncière et stratégies patrimoniales.**

Accompagnement des grands propriétaires fonciers dans la valorisation et l'optimisation de leurs actifs, proposant notamment la réalisation des études de valorisation et des missions d'appui à la transaction.



Programmation foncière et immobilière



Développement économique et appui aux industriels



Reconversion de site et transaction immobilière



Appui au montage de projets urbains et d'aménagement

Fonctionnement de l'entreprise

Pour se placer sur le marché, MODAAL propose ces services sur des demandes d'études qui leur correspondent, ou les attentes du client seront remplies grâce aux compétences de ces équipes. MODAAL rentre donc en compétition avec d'autres sociétés concurrentes pour proposer les meilleures solutions et contrats, principalement en répondant à des appels d'offres.

Une fois un contrat gagné, MODAAL va tout mettre en œuvre pour proposer le meilleur résultat d'étude possible et pour se faire la société est divisée en **deux parties** :

- Un **pôle-conseil** composé d'architectes, d'urbanistes et de programmistes qui vont réaliser des missions d'analyse de territoire et des études de terrains complètes par rapport aux demandes des clients. Leurs buts vont être de répondre aux besoins et aux questions en fournissant des présentations concrètes et complètes.
- Un **pôle Data / SIG** (Système d'Information Géographique) composé de géomaticiens et développeurs vont appuyer les consultants en leur fournissant des données et des cartes propres et lisibles sur les terrains ou territoires étudiés. Le pôle Data / SIG s'occupe aussi de la création d'outil web sur la demande de client ou la création de projet au sein de la société.

Organisation MODAAL

Edouard Mouglin
Associé



Yann Milon
Associé-fondateur



Justine Monéreau
Office Manager



Guillaume Hameau



Fabien Marty
Ingénieur
Développement
Responsable applicatif

DATA - SIG

GONON	Léonie
HENNENFENT	Lou
RATURAT	Grégoire



Hilda Draye
Responsable du
pôle Etudes
territoriales

Etudes Territoriales

CHAPUIS	Quentin
PREVOT	Inès
LEROY	Léo
MATULA	Laury-Anne



Etienne Terraillon
Architecte,
responsable du
pôle faisabilité

Faisabilité architecturale et urbaine

PATIN	Anne
REMY	Laurène
ROUAUD	Tom
VEYSSET	Clémentine

Montage, économie de projet

ROSSIGNOL	Agnès
-----------	-------



Stagiaire :
Claire
Bouchard



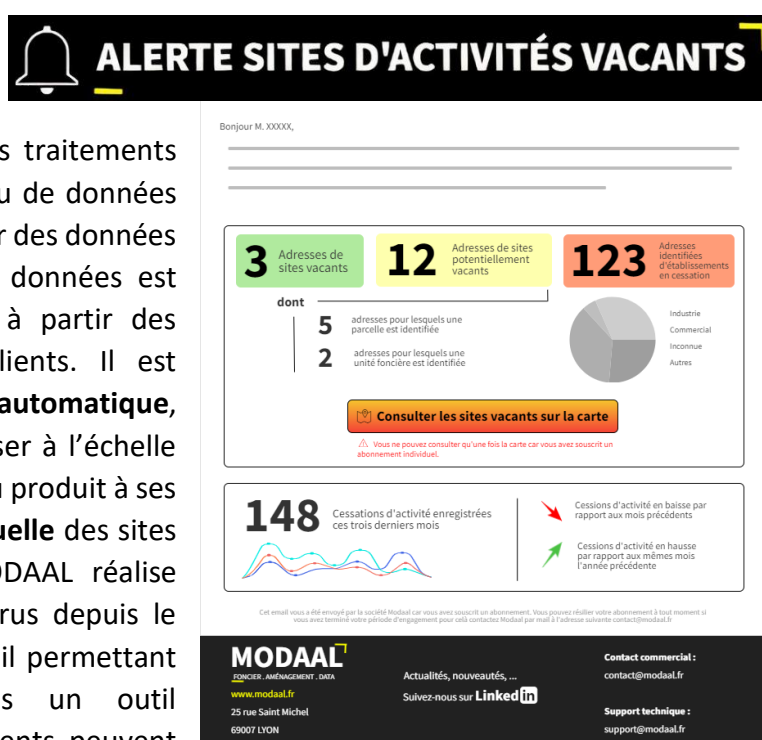
Stagiaire :
Inès
Lancelin



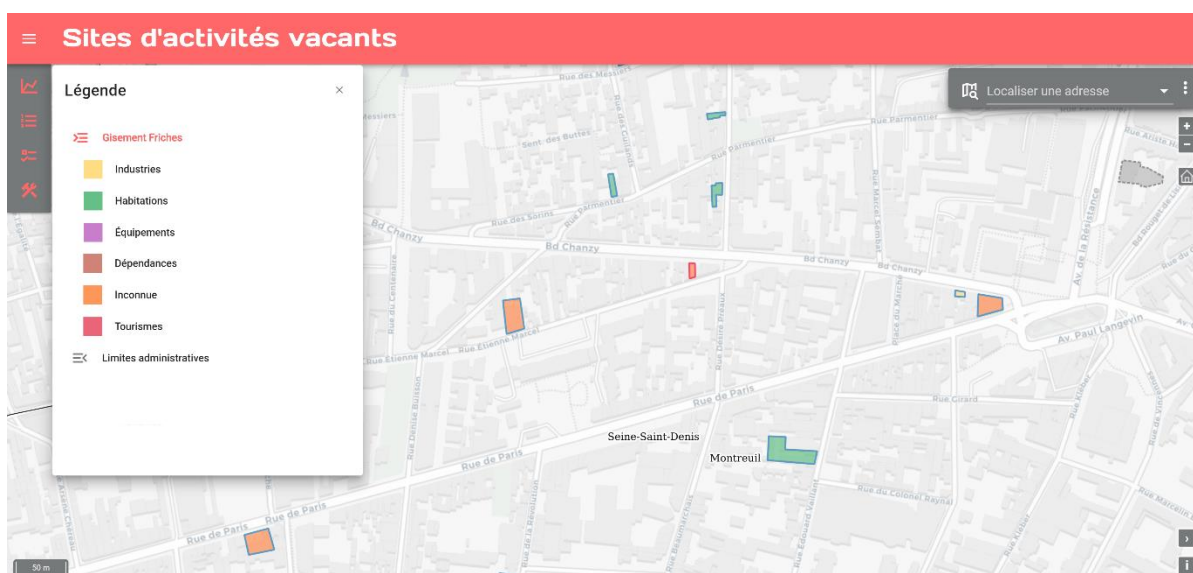
Aujourd'hui MODAAL connaît une croissance de son activité et du nombre de ses collaborateurs ce qui lui permet d'investir dans des projets de recherches et développement. Ainsi, ses équipes travaillent sur le **développement d'outils** pour améliorer ses analyses, diagnostics et proposer à ses clients des solutions innovantes pour répondre à des problématiques nouvelles. Les applications, produites par MODAAL, permettent une visualisation des données sur des cartes dynamiques et une centralisation des données en [open data](#).

Outils Web

MODAAL a investi sur ces traitements de données pour proposer un jeu de données des sites d'activité vacants à partir des données [Sirene](#) et [Infogreffe](#). Ce jeu de données est amélioré de manière continue à partir des missions réalisées pour nos clients. Il est maintenant généré de manière **automatique**, ce qui nous permet de le proposer à l'échelle nationale et proposer un nouveau produit à ses clients : **le service d'alerte mensuelle** des sites d'activité devenus vacants. MODAAL réalise une qualification des sites apparus depuis le mois précédent et envoie un mail permettant de consulter les sites dans un outil cartographique en ligne. Nos clients peuvent donc être très **réactifs** sur du foncier qui se libère et peuvent maintenir leur gisement de friches à jour.



Email nouveaux sites d'activités vacants

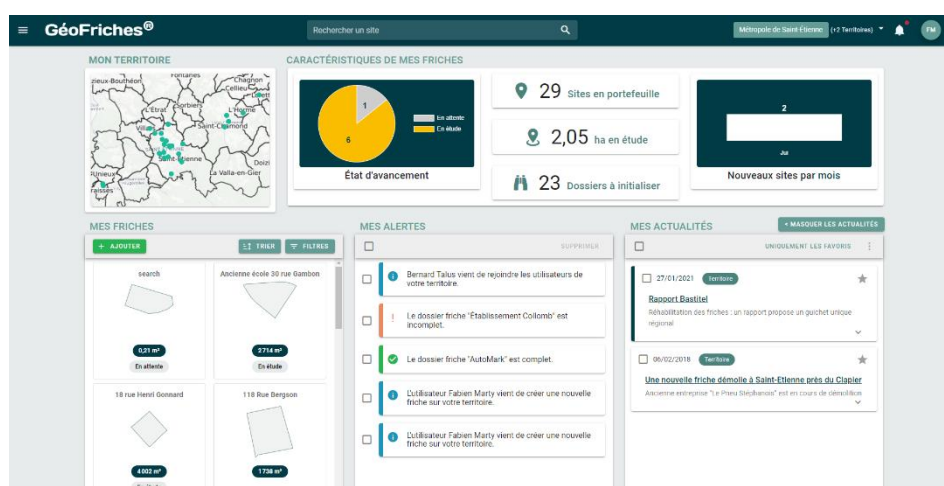


Carte interactive pour la visualisation des sites détectés

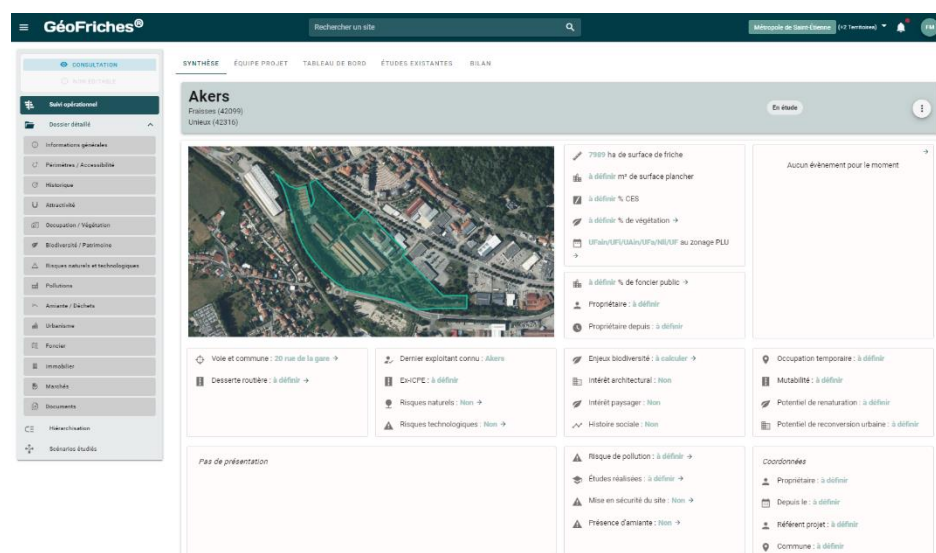
GéoFriches® une application trouvant son origine dans le « **paradoxe foncier** » qui a été constaté sur de nombreux territoires : d'un côté, **une pénurie de foncier** pour répondre aux besoins de logements, d'équipements, d'accueil d'entreprises, mais aussi la présence de **nombreuses friches** (terrains abandonnés) en attente de recyclage. Concrètement, c'est une application web à destination des acteurs publics et privés qui facilite et accompagne la reconquête des friches.

Modules de GéoFriches® :

- **Recenser** facilement les friches : Observer et prospecter sur les territoires d'intervention.
- **Qualification** : Qualifier les sites pour choisir les meilleurs scénarios de reconversion.
- **Hiérarchiser** et prioriser les sites : Créer et définir des grilles d'analyses multicritères pour hiérarchiser les friches.
- **Scénariser** la remobilisation : Facilite la prise de décision en définissant des scénarios différents pour des réhabilitations.
- **Piloter** et évaluer leur reconversion : Planifier et évaluer les actions de reconversion de friches.



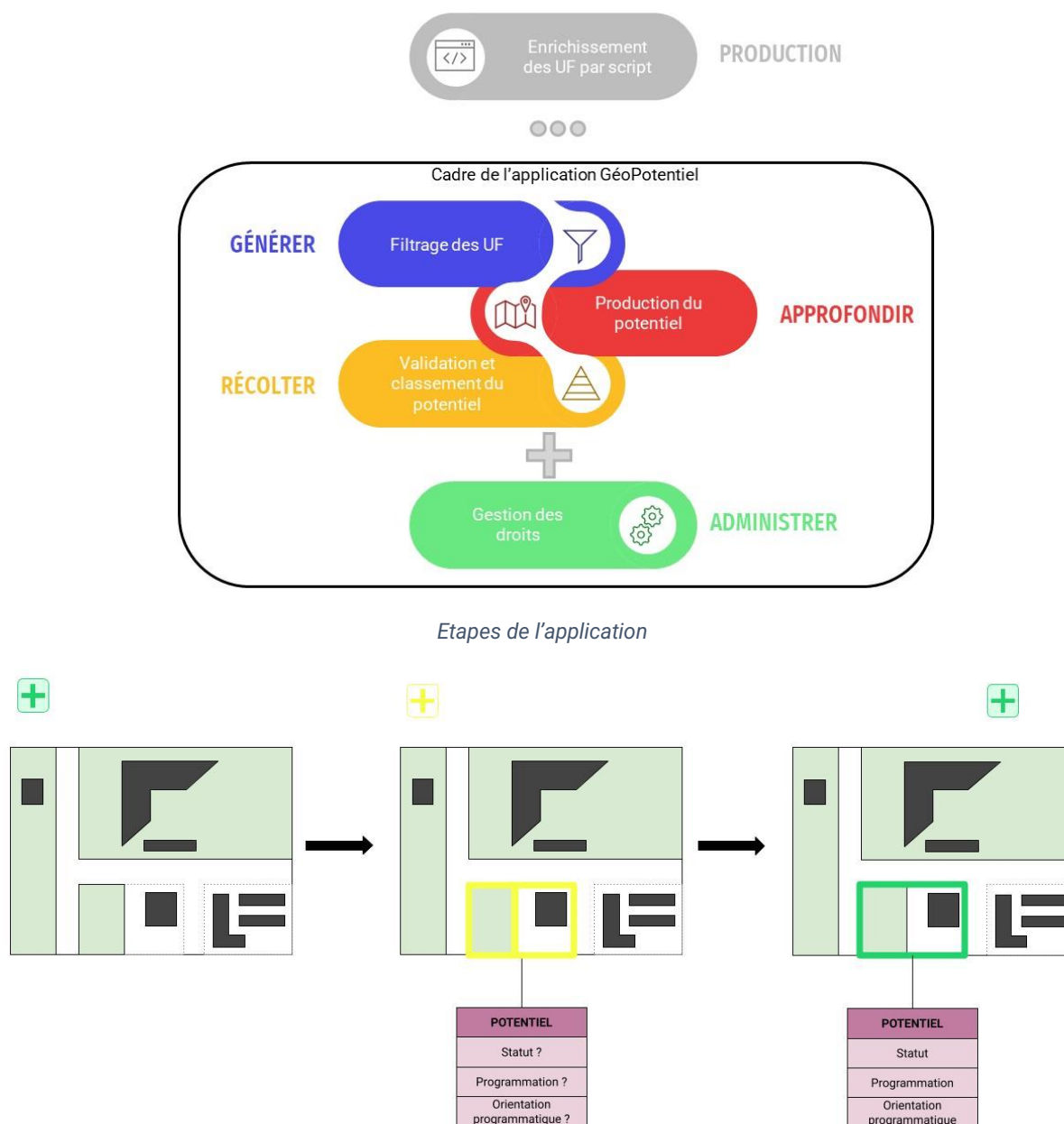
Page d'accueil d'un utilisateur connecté



Dossier de qualification d'un site

GéoGisement®

GéoGisement®, nouvellement appelé **GéoPotentiel®**, une application qui a pour but de faciliter des méthodes de travail qui sont aujourd'hui **chronophages** et qui présentent des **risques de perte de données** lors de la qualification des sites de potentiel foncier. En effet, le temps de production et la **répétitivité** du travail à certaines étapes du projet peuvent représenter un véritable inconvénient lors des échanges entre MODAAL et ses clients. Le **potentiel foncier** correspond aux **possibilités de développement** et **d'utilisation d'un terrain**, en fonction des contraintes et des opportunités du contexte environnant. Par exemple, en identifiant les unités foncières d'une parcelle connue pour être en friche, il devient possible de les regrouper et de les traiter de manière collective.



Génération d'un potentiel avec la détection d'unité foncière

Missions

Dès lors de mon arrivée, j'ai eu besoin d'une phase d'adaptation assez importante à un milieu qui m'était totalement étranger. Énormément de concepts m'étaient inconnus et même aujourd'hui certains restent flous. L'**urbanisme** est un milieu complexe avec beaucoup de facettes, associées à l'**architecture**, la **géographie**, l'**aménagement**, l'**économie**, l'**écologie**, mais aussi la **politique et les réglementations**. Le nombre de champs d'étude est très large, mais découle principalement des **politiques urbaines** (logement, économie, transport, et environnement) et des **planifications urbaines** pour maximiser le potentiel géographique.

« Ensemble des sciences, des techniques et des arts relatifs à l'organisation et à l'aménagement des espaces urbains, en vue d'assurer le bien-être de l'homme et d'améliorer les rapports sociaux en préservant l'environnement. » [Centre national de ressources textuelles et lexicales du CNRS \(CNRTL\)](#)

GéoFriches

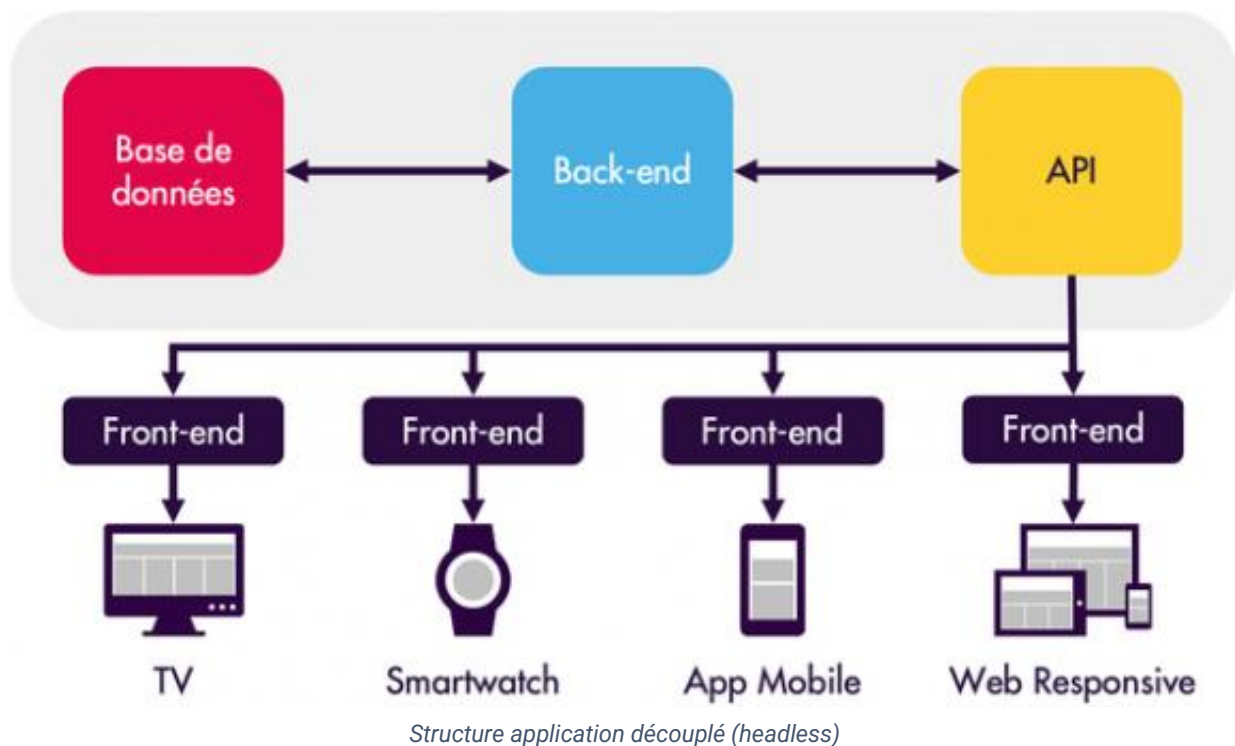
L'application GéoFriches® est développée en collaboration avec **TESORA**, un cabinet d'études spécialisé dans le domaine des sites et sols pollués. Du côté de TESORA, l'application est portée par **PAUGET Benjamin** (Responsable R&D) et **BENCHERIF Djamel** (Développeur Web). En ce qui concerne MODAAL, **MOUGIN Edouard** (Associé) est responsable du projet, avec l'aide de **MARTY Fabien** en tant que développeur. Nous sommes donc trois développeurs travaillant sur l'application, ce qui nécessite une approche de **travail d'équipe** sur le projet. Pour faciliter cette collaboration, nous utilisons [GitLab](#) pour partager le code source et assurer un suivi efficace du projet.

Technologies et structure de l'application

L'application GéoFriches® utilise différentes technologies adaptées en fonction des besoins de l'application. L'une des premières solutions évoquées pour répondre au mieux à ce besoin était le fait de **découpler** l'application web, utiliser un **système headless** (pas directement relié au front-end = head), c'est-à-dire permettre la gestion du **front-end** (partie visible par l'utilisateur) et du **back-end** (partie permettant la liaison à la base de données) de façon séparée.

Cela a pour avantage :

- Donner la **liberté d'utilisation de langage** différent sur les deux parties de l'application
- **Faciliter la maintenance** du site Web. Deux petites applications ont moins d'**effets de bord** (effets secondaires) qu'une grande application.
- Réutilisation de la partie [API](#) du back-end pour **différentes surfaces** (desktop, mobile)
- Faciliter l'utilisation de [framework](#) pour le front



L'application ayant pour but de **faciliter l'étude de terrain** et de foncier, il est obligatoire d'être capable de gérer des **données spatiales complexes** et de permettre leurs affichages avec des **cartes interactives fluides**.



Concernant l'utilisation et le traitement de ces types de données, le choix s'est tourné vers une [base de données relationnelle](#) permettant de stocker de manière ordonnée une multitude d'informations dans différentes tables tout en gardant une liaison entre toutes ces informations. **PostgreSQL** est le choix parfait, puisqu'il est capable de gérer des bases de données complexes de façon fiable et robuste. Il offre aussi une grande diversité de type de données, en prenant en charge des vidéos, des images, des fichiers XML et le plus important pour nous, la gestion de données spatiale grâce au plugin **PostGis**.



La base de données en place, la partie back-end doit être capable de récupérer et traiter les informations disponibles pour pouvoir les servir au front. Pour la conception du back-end, il s'agit de **Node.js**, un environnement d'exécution [single-thread](#), [open-source](#) et **multiplateforme** permettant de créer des applications rapides et évolutives côté serveur et en réseau. Il utilise une [architecture d'E / S \(entré / sortie\) non bloquante](#) et pilotée par les événements, ce qui le rend efficace et adapté aux applications en temps réel.



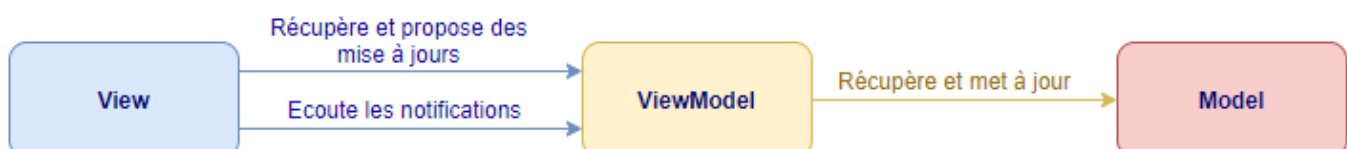
Pour récupérer les informations depuis la base de données, nous utilisons un [ORM](#) pour améliorer la vitesse de développement. L'ORM **Sequelize** prend en charge **PostgreSQL** et permet de générer les tables de la base via des modèles en **JavaScript**, ainsi que les relations pouvant relier les tables générées. Il offre la possibilité d'exécuter des opérations **CRUD** (Create Read Update Delete) sur les enregistrements, les données passeront par une phase de validation avant d'être envoyées dans la base lors de l'utilisation de Create (Créer).

Express JS

La mise en place de l'**api** est nécessaire pour permettre au front de récupérer les informations à afficher. Pour cela, nous utilisons **Express JS**, qui offre une couche d'abstraction au-dessus du **moteur http de NodeJS**, permettant ainsi d'écouter l'arrivée de paquet (requête) sur des chemins (Route) grâce à différentes url et de répondre en envoyant la réponse attendue, qui est généré grâce à des méthodes utilisant **Sequelize** pour fournir les données.

Concernant la partie visible par les utilisateurs, le **front-end**, nous sommes restés sur le même environnement que le back-end, **NodeJS**. Il fournit une multitude de bibliothèques simplifiant le développement, dont le framework **VueJS** qui permet de créer des **interfaces Web interactives**, lors du commencement du développement de l'application. VueJS était en version deux, depuis la version trois est sortie, le passage à la nouvelle version sera une mise à jour future qui apportera son lot de changement important. VueJS se concentre sur la **couche ViewModel du modèle MVVM**.

Le modèle **MVVM** (Model View ViewModel) en architecture logicielle est un modèle d'**architecture logicielle** (design pattern) utilisé pour faciliter la conception des applications, en séparant la logique en **trois couches**.



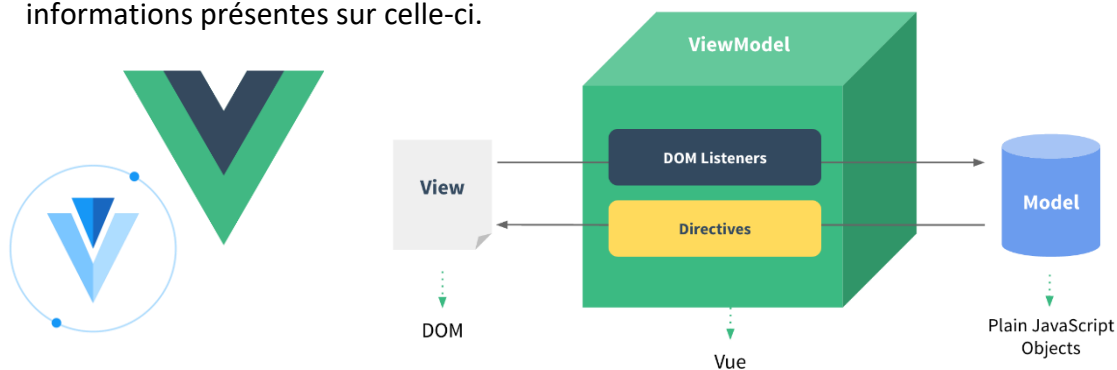
Le modèle se base sur les trois composants suivants :

- **Modèle** (Model) : Le modèle représente les **données** et la **logique métier** de l'application. Cela peut inclure des classes qui définissent les objets de données, des

opérations de récupération et de manipulation des données, ainsi que d'autres fonctions liées à la logique métier.

- **Vue (View)** : Une vue est une **description d'une interface graphique**. Elle établit la connexion entre les actions de l'utilisateur et le modèle de vue. Elle définit où et comment les composants graphiques sont placés sur l'interface et décrit les liaisons de données entre les valeurs affichées et le modèle de vue.
- **VueModèle (ViewModel)** : Le modèle de vue est chargé de transformer et **d'organiser le modèle métier** pour définir ce que la vue doit afficher à l'utilisateur. S'il y a des changements importants au niveau du modèle métier, le modèle de vue peut être ajusté sans affecter la vue.

Le modèle de vue permet de définir une **séparation nette** entre le **modèle** et la **vue**, ce qui se traduit par une meilleure gestion de la logique métier. Il permet de définir des **tests unitaires** plus faciles et offre une plus grande **réutilisation du code**. Un aspect important, que permet ce modèle, est de permettre l'affichage de **données réactives** pour l'utilisateur, c'est-à-dire qu'il n'est pas nécessaire recharger complètement la page pour modifier des informations présentes sur celle-ci.



Pour faciliter le développement de l'interface utilisateur, nous utilisons un **framework** de composants basés sur la spécification [Material Design](#), fournissant une **collection composants** tels que des boutons, des tableaux et bien d'autres, qui sont directement prêts à l'usage. Il s'agit de **Vuetify** qui est construit au-dessus de **VueJS**, open source et qui offre des composants modifiables. Il est l'outil parfait pour accélérer le déploiement d'une application puisqu'il suffit d'ajouter directement ces composants préconçus dans les nouvelles pages. Il n'est pas nécessaire de tout développer de zéro.

Dessert (100g serving)	Calories	Fat (g)	Carbs (g)	Protein (g)	Iron (%)
Frozen Yogurt	159	6	24	4	1
Ice cream sandwich	237	9	37	4.3	1
Eclair	262	16	23	6	7
Cupcake	305	3.7	67	4.3	8
Gingerbread	356	16	49	3.9	16

Exemple du composant table fournit par Vuetify



La nécessité d'afficher des **données spatiales** implique l'utilisation d'une bibliothèque permettant l'intégration de **cartes interactives** dans un environnement web. **Openlayers** est une bibliothèque JavaScript intégré avec NodeJs qui offre énormément de fonctionnalités cartographiques tel l'affichage de couches de données provenant de multiples sources, avec de multiples formats, la représentation de ces couches, l'écoute d'événements et bien d'autres.



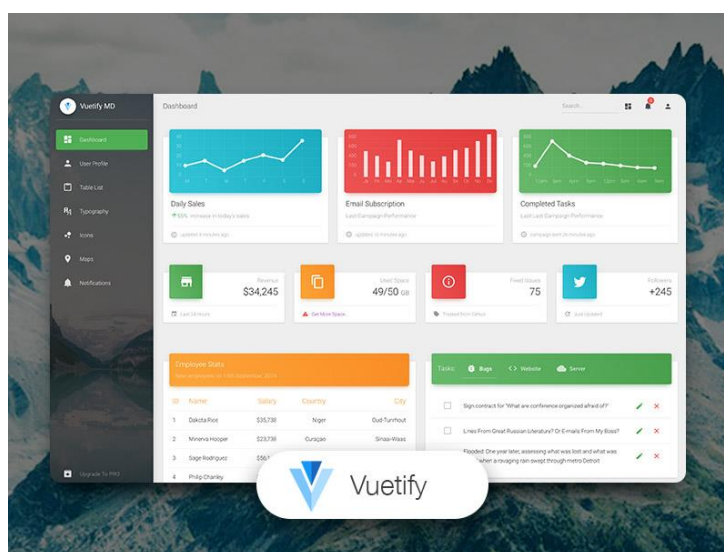
Certaines données que l'on souhaite afficher sur les cartes générées par **Openlayers** sont stockées sur les serveurs de MODAAL, puisqu'elles sont retravaillées pour être améliorées. Il est donc nécessaire de les publier sous forme de service. Pour cela, nous utilisons le serveur cartographique **GeoServer**, un serveur web open source permettant de partager et modifier **des données géographiques**. Il permet de publier les données provenant **de source spatiale** utilisant les normes de publications pour ce type de données, notamment les standards de l'[Open Geospatial Consortium](#) (OGC), qui est une organisation internationale à but non lucratif. Elle a pour but de promouvoir des standards pour garantir l'interopérabilité dans le domaine de la géomatique.

Un schéma est disponible en [annexe](#) pour décrire l'architecture entre ces différentes technologies.

Fonctionnement du Front-End

La partie **front-end** constitue la **partie visible** par les utilisateurs. Pour le développement, comme mentionné précédemment, nous allons utiliser le **framework Vuetify** qui donne accès à une vaste gamme de **composants** facilitant et accélérant le processus de création. C'est un projet **open source** qui propose des templates fournis par la **communauté**, ce qui implique sa pérennité ainsi qu'une documentation détaillée.

Vuetify possède une panoplie **d'éléments préconstruits** tels que des boutons, des barres de navigation interactives, des cartes dynamiques, et bien d'autres. Cela nous permet de créer rapidement des **interfaces utilisateur modernes et ergonomiques**. Il nous permet de consacrer davantage de temps à **l'optimisation des fonctionnalités** et à la qualité de l'expérience utilisateur.



Exemple de composant disponible

Les informations qui seront affichées à l'écran proviennent de l'**API** de notre backend et sont **stockées** dans notre navigateur grâce au principe de « **store** » expliqué précédemment dans ce document. Un store adapté à chaque partie de l'application est nécessaire pour une **gestion optimale** des pages. Grâce à **Vuex et Vue**, nous pouvons rendre nos **pages réactives**, leur permettant de s'adapter automatiquement aux changements de données présentes dans le store.

Concernant le contenu de la page, nous sommes guidés par des **templates** réalisés par les chefs de projet sur des **diapositives PowerPoint**. Ces diapositives sont créées au fur et à mesure de nos avancements dans le développement.

L'absence **de cahier des charges** complet au début du projet a pu poser **quelques problèmes**. Le principal défi lié à ce manque de spécification était **l'incertitude** quant aux attentes spécifiques et aux fonctionnalités essentielles de la page. En l'absence de directives détaillées, nous avons dû faire des **hypothèses** sur certains aspects du design et des fonctionnalités, ce qui a entraîné des **allers-**



retours fréquents avec les chefs de projet pour valider nos choix. Nous avons parfois **épuisé les templates** existants, ce qui a **ralenti** notre progression. De plus, les modifications ultérieures des templates ont entraîné des **ajustements** et des **incohérences**.

Grâce à une **collaboration continue** et à une **approche flexible**, nous avons réussi à **surmonter** ces obstacles liés au manque initial de cahier des charges. Nous avons démontré **l'importance d'un cahier des charges détaillé** dès le début du projet, ce qui aujourd'hui porte ses fruits notamment pour le développement de l'application **GéoPotentiel** de MODAAL.

Fonctionnement du Back-End

Le **back-end** d'une application web est la partie responsable de la **gestion des données** et des **interactions côté serveur**. Lorsque le back-end reçoit une requête, il interroge la base de données ou interagit avec, pour ensuite renvoyer la réponse formatée pour le front-end. Le back-end constitue le **cœur fonctionnel de l'application**, tout en restant invisible pour l'utilisateur.

Express.js, framework utilisé pour gérer le serveur back-end, fonctionne sur le principe de **client-serveur** pour gérer les requêtes des utilisateurs et fournir des réponses en fonction. Lorsqu'un utilisateur arrive sur **app.geofriches.fr**, son navigateur envoie une **requête HTTP** au serveur back-end. Lorsque le serveur express tourne, il **écoute** sur une **adresse IP** et un **port** précis. Dès qu'il reçoit une requête depuis cette adresse, il répond en fonction de la configuration qui lui est attribuée.

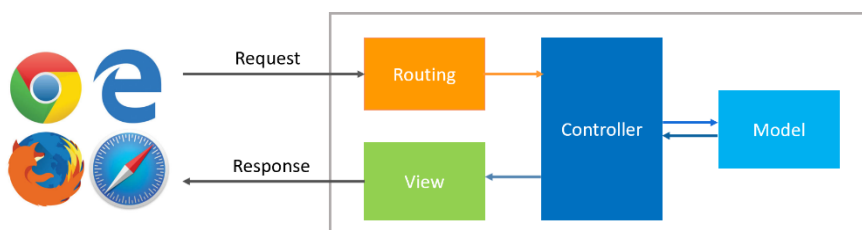
Il existe plusieurs **types de requêtes** que le serveur peut recevoir :

- **Get** : signifiant « obtenir », une requête GET **demande** au serveur de lui fournir des **ressources spécifiques**. Le navigateur récupère ensuite toutes les données de la page web demandée et les affiche à l'écran.
- **Post** : le navigateur **envoie des données au serveur**. Cette méthode est principalement utilisée pour envoyer des données de formulaire HTML, lorsque les internautes remplissent un formulaire sur un site web.
- **Delete** : requête permettant la **suppression de ressource**.
- **Put** : similaire aux méthodes précédentes, permet également de transférer des données vers le serveur. Son utilisation vise à **ajouter ou remplacer une ressource spécifique** sur le serveur.



Line opération CRUD et requête HTTP

Dans une application, il est plus optimisé d'avoir un **GET par ressource** pour éviter de devoir retourner toutes les informations d'un coup, pour cela, il est donc nécessaire que le serveur comprenne vers quelle méthode il doit retourner la requête. Pour ce faire, il redirige grâce à un **routing**.



Principe de routage ExpressJs

Le routage désigne le **processus de sélection des chemins dans un réseau** pour acheminer les données d'un expéditeur vers son ou ses destinataires. Dans le cas d'express, la notion de route permet de **distinguer le traitement des requêtes** en fonction de la méthode utilisée, du chemin demandé et des données éventuellement incluses dans la requête. Chaque route établit une correspondance entre les caractéristiques d'une requête et le traitement spécifique qui lui est associé.

Le serveur reçoit une requête via l'une de ses routes et la traite à l'aide du **contrôleur** correspondant à la route demandée. Une fois le traitement terminé, le serveur renvoie une réponse au client en utilisant le **protocole de communication** aller-retour, HTTP.

La réponse du serveur peut être variée, dans notre cas, il s'agit de données **JSON** permettant l'affichage des informations sur l'application front-end (navigateur). Ainsi, Express.js suit cette **approche classique client-serveur** pour gérer les interactions entre les utilisateurs et les applications web.

```
const router = require("express").Router()
const communeController = require("../controllers/geofriches/commune")

router.get("/region/", communeController.getAllRegion)
router.get("/department/:id", communeController.getRegionDepartment)
router.get("/epci/:id", communeController.getDepartmentEpci)
router.get("/", communeController.getAll)

module.exports = router
```

Exemple de routage, avec paramètre (Communes)

Pour simplifier le développement du back-end de l'application, qui correspond à une **API** permettant d'interagir avec la base de données, nous utilisons un **ORM** nommé **Sequelize**. L'utilisation d'un ORM permet de simplifier grandement le processus de développement en fournissant des outils de gestion de base de données prêt et complet. Pour que l'ORM fonctionne, il est nécessaire dans un premier temps de faire la connexion à notre base de données :

```
const env = process.env.NODE_ENV || "production"
const config = require(`${__dirname}/../config/database.js`)[env]
const db = {}

const sequelize = new Sequelize(config.database, config.username, config.password, config)
```

Il est nécessaire de définir une configuration de connexion adaptée à la phase de développement et la production distincte afin de séparer les espaces de stockages. En **développement**, la **base de données** est en **local**, facilitant le développement et **limitant** la **propagation d'erreur** dans la base.

Maintenant connecter, nous pouvons utiliser notre ORM dans toute sa puissance, nous allons commencer par générer des **migrations** pour permettre la mise en place de base données précédemment structurées.

Les migrations dans **Sequelize** permettent de **suivre les modifications apportées à la base** de données, tout comme les **systèmes de contrôle** de version. Ces **transitions d'état** sont enregistrées dans des fichiers de migration, qui décrivent comment accéder au nouvel **état** et comment annuler les modifications pour revenir à l'ancien état. Une migration dans Sequelize est un **fichier JavaScript** qui exporte deux fonctions, « **up** » et « **down** », définissant comment effectuer la migration et la rétrograder. Ces fonctions seront automatiquement exécutées au démarrage du serveur.

```
"use strict"

module.exports = {
  ...
  async up(queryInterface, Sequelize) {
    await queryInterface.changeColumn(
      { schema: "qualify", tableName: "urban_zoning_doc" },
      "producer",
      {
        type: Sequelize.STRING(100),
      }
    )
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.changeColumn(
      { schema: "qualify", tableName: "urban_zoning_doc" },
      "producer",
      {
        type: Sequelize.STRING(50),
      }
    )
  },
}
```

Exemple de migration modifiant le nombre de caractère d'un champ

Maintenant la base de données créée, il est nécessaire de fournir au back-end la structure des informations contenues dans la base, permettant ainsi de ne pas envoyer n'importe quoi dans la base de données. Un modèle **Sequelize** établit la **correspondance** entre **la table SQL** et **l'objet JavaScript** que nous allons définir dans un fichier JS et généralement stocké dans le dossier *models*. Le modèle généré pour la table Commune est disponible en [annexe](#).

Ensuite, il faut définir qu'elles seront les **interactions** possibles avec les données disponibles en base, et pour chacune d'entre elles définir **un chemin** sur la route qui renvoie vers la **méthode** permettant d'interagir avec la base de données. Cette méthode est contenue dans un fichier que l'on nommera **contrôleur** (controller). Dans l'exemple de routage présenté plus haut, on peut voir la définition de la route des communes qui fait appel, pour chaque route définie, à la méthode du contrôleur. Par exemple si Express reçoit une requête Get avec l'url '*<cheminVersLeServeur>/commune/region*', il rédigera la requête vers la méthode « *getAllRegion* » contenue dans le fichier des contrôleurs de Commune importé.


```
const { Department, Epci } = require("../models")
const ErrorResponse = require("../utils/Error_utils")

exports.getRegionDepartment = async (req, res, next) => {
  try {
    const departments = await Department.findAll({
      where: { insee_reg: req.params.id },
      order: [["name_dep", "ASC"]],
    })
    if (departments) {
      res.status(200).json({ success: true, departments })
    } else {
      res.status(404).json({
        errors: [{ msg: "Pas de régions" }],
      })
    }
  } catch (e) {
    next(new ErrorResponse("error server", 500))
  }
}

exports.getDepartmentEpci = async (req, res, next) => {
  try {
    const dep = await Department.findAll({
      where: { insee_dep: req.params.id },
      include: [Epci],
      order: [[Epci, "name_epci", "ASC"]],
    })
    if (dep) {
      res.status(200).json({ success: true, epcis: dep[0].Epcis })
    } else {
      res.status(404).json({
        errors: [{ msg: "Pas de régions" }],
      })
    }
  } catch (e) {
    next(new ErrorResponse("error server", 500))
  }
}
```

On peut remarquer que grâce à l'ORM, lorsque l'on veut interroger la base de données, nous n'avons pas besoin d'écrire de **requêtes SQL longues et complexes**. **Sequelize** nous offre des **méthodes** permettant **d'interroger la base** avec de multiples options. Dans cet exemple, nous allons chercher les départements en fonction de la région, grâce à l'option **WHERE**, on peut lui définir le code INSEE de la région contenu dans les paramètres de la requête reçu par le serveur. On lui demande aussi d'ordonner le résultat dans l'ordre croissant (ASC) des noms de départements.

Déploiement de l'API BAN

Le besoin (Géocodage)

La **Base Adresse Nationale** est l'une des neuf bases de données du [service public des données de référence](#). Elle est la seule base de données d'**adresses officiellement reconnue** par l'administration. Elle permet donc de vérifier si une adresse **existe réellement** grâce au **géocodage**, nous utilisons l'API fournit en ligne pour nettoyer toutes les adresses incorrectes des bases de données libres de droit, tel que **SIRENE** (Système national d'identification et du répertoire des entreprises et de leurs établissements). L'API retourne une adresse avec un libellé correct, complet et la position géographique associée à l'adresse, même à partir d'une saisie incomplète.



Mais il est arrivé un moment où nos demandes étaient trop importantes ou **trop volumineuses** pour l'API BAN mise en ligne par les services de l'état, qui finissait par nous **bloquer**. On se retrouvait dans l'impossibilité d'utiliser l'API pendant un certain moment, ce qui nous **bloquait** dans l'avancement de nos traitements de données.

La solution

Après quelques recherches, nous avons fini par trouver un moyen d'installer l'API BAN en local grâce à l'utilisation de docker et **ADDOK**. ADDOK est un **moteur de géocodage open source** élaboré par **Etalab**. Le projet ADDOK est accompagné d'une **documentation détaillée** concernant l'installation du logiciel et son approvisionnement via la Base Adresse Nationale (BAN). ADDOK a été conçu spécifiquement pour les adresses, offrant une meilleure **performance** que des outils de recherche génériques tels qu'[Elasticsearch](#) pour ce type de

BaseAdresseNationale/
addok-docker

Conteneurs Addok pour Docker



3
Contributors

3
Issues

111
Stars

19
Forks



tâche. Le code est open-source et est **disponible sur GitHub**, permettant aux utilisateurs de déployer leur propre instance d'ADDOK.

Le processus de géocodage avec ADDOK passe par **deux étapes** : la **collecte d'adresse possible**, pour ensuite les évaluer en les comparant au texte initial cherché. ADDOK va créer des **index** correspondant à des **mots simplifiés phonétiquement**. Les index les moins courants sont traités en priorité pour améliorer la rapidité de réponse.

Après la collecte des adresses possibles, il **compare les libellés** complets des adresses de référence avec celui cherché, y compris le numéro de rue. Cette comparaison utilise des **trigrammes** (une séquence de trois caractères consécutifs), pour comparer des noms de lieux, en ignorant l'ordre des caractères, ce qui permet de trouver des similitudes malgré des variations. Il attribue un score de comparaison en fonction de ses recherches précédentes, sachant que le score peut être influencé par des critères géographiques, tel que la population.

Le moteur d'ADDOK utilise **Redis** (REmote DIctionary Server), un **système de gestion de base de données** clé-valeur en mémoire. Redis permet un **accès rapide aux données**, car elles sont stockées en **RAM** (mémoire vive) plutôt que sur un disque dur, ce qui permet des opérations de lecture et d'écriture **plus rapides**.



Le déploiement

Pour installer l'API sur un serveur, la procédure utilise **Docker**, puisque la solution consiste à télécharger le **conteneur** pour inclure ADDOK avec les données de références diffusées par la [Base Adresse Nationale](#). **Docker** est une **plateforme de conteneurs**. Elle permet de créer facilement des conteneurs et des applications inclus dans ces **conteneurs**. C'est une solution **open source**, **sécurisée** et **économique**, tout en étant facile à déployer et à utiliser. Un conteneur est un **environnement d'exécution** léger, **alternative** aux méthodes de **virtualisation traditionnelles** basées sur les **machines virtuelles**. Etant plus léger, il offre des performances bien plus intéressantes qu'une machine virtuelle qui nécessite d'émuler un environnement.

Pour pouvoir accéder à l'API quand on le souhaite, il était nécessaire **d'héberger** le conteneur créé par **Docker** et le code fourni par le [GitHub BaseAdresseNationale/addok-docker](#). La solution la plus optimale dans notre contexte, consistait à utiliser un **hébergement d'OVH Public Cloud** qui offre un service d'hébergement de ressources à la demande, c'est-à-dire que la **solution cloud** n'est active que lorsque c'est nécessaire. Le système implique donc « **pay-as-you-go** », seules les ressources utilisées seront payées. L'API est donc **disponible** que lorsque cela est **nécessaire**, puis désactivée directement depuis le script qui tourne.

Usage

Une fois le service mis en place, ainsi que la **documentation** créée, nous avons dû ajuster les **ressources** disponibles au serveur pour que l'API puisse tourner efficacement. Nous sommes donc rentrés dans **une phase de tests** concernant les capacités de notre installation à nous retourner le plus rapidement possible les données demandées.

Même une fois l'API BAN installée sur notre serveur, nous avons toujours un problème **de volume de données à traiter**. Pour résoudre ce problème, nous devons envoyer les adresses par lots sous forme de fichier JSON. Nous avons donc **divisé par morceaux** de taille définis (**chunk**) le nombre d'adresses à envoyer à l'API et bouclé dessus pour avoir la réponse pour toutes nos données. C'est de cette manière, que nous pouvions traiter le plus **rapidement** possiblement les adresses qui avaient un besoin de vérification, tout en évitant de se faire bloquer par la quantité de requêtes lourdes envoyées.

Le script python utilisant la méthode de traitement par lot est disponible en [annexe](#).



Script Python

Python est le langage préféré des développeurs d'après une étude réalisée par [Jetbrains](#), selon l'index **Tiobe**, il est le langage le plus populaire avec [13,45%](#) des votes. La raison de sa **popularité** peut venir de la **puissance** qu'il est capable de fournir, ou bien de sa **simplicité** et **facilité d'apprentissage**. Il trouve toute sa puissance dans la **gestion d'infrastructure**, **l'analyse de données**, le développement de logiciels et de nos jours le développement d'intelligence artificielle. Une multitude de **bibliothèques préconstruites** sont offertes par Python, permettant aux développeurs de se concentrer sur leurs tâches plutôt que sur les détails techniques.

Python possède une **syntaxe bien plus simple** à prendre en main par les développeurs que d'autres langages, rendant le processus de développement plus rapide. Grâce à cette approche, les informaticiens peuvent se concentrer davantage sur leur travail plutôt que sur la manière de le mettre en œuvre. Python est donc très accessible pour les **débutants** mais nécessite quand même un plus grand **investissement** lorsque l'on veut approfondir les pratiques. En comparaison avec d'autres langages, la syntaxe de Python est moins lourde et plus flexible.

La popularité de python a entraîné la prise en charge par de **nombreux systèmes d'exploitation**. Ainsi qu'une vaste gamme de **bibliothèques** et d'**API** compatibles est disponible, ce qui accroît sa polyvalence. C'est un langage **dynamique** qui reste toujours à jour grâce aux mises à jour régulières de l'équipe de développeurs. La popularité du langage induit aussi une communauté active qui peut fournir de l'aide et des solutions pour résoudre des problèmes techniques. Python est donc adapté aussi bien au **scripting** et à **l'automatisation** qu'au développement de **logiciels professionnels**.

Un **défaut principal** que l'on peut reprocher à Python réside dans sa **lenteur** par rapport à certains autres langages tels que **C** ou **Java**. Pour **optimiser les performances**, certaines manipulations de données peuvent être accélérées en utilisant des bibliothèques telles que **NumPy** et **Pandas**.



Gestion d'un risque

La perte de ces informations peut entraîner des **temps d'arrêt prolongés**, puisqu'il devient nécessaire de les **restaurer manuellement**. Cette procédure fait perdre un temps précieux aux équipes et peut **nuire** considérablement à la **productivité** de l'entreprise. Cependant, ce problème ne se pose que si la récupération des données est encore possible. Si aucune sauvegarde n'est disponible, l'entreprise risque de subir des **conséquences plus graves** et de se retrouver en difficulté majeure.

L'absence de sauvegarde de données peut engendrer un **risque de sécurité**, puisque si la base de données subies une cyberattaque, ou pire si le centre de données prend feu, les données utilisateurs peuvent se retrouver **compromises**, devenir **inaccessibles** ou être totalement **détruites**. La réputation de l'entreprise à la suite des conséquences pourrait en pâtir, elle perdrait la confiance des clients et montrerait un manque de conformité aux réglementations en matière de **protection des données**.

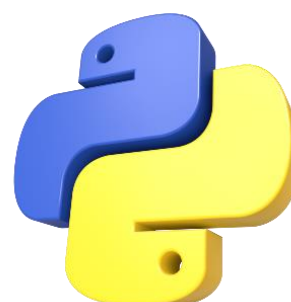




L'**identification des risques** a permis de définir une stratégie de gestion de ces risques afin d'éliminer, ou réduire, leurs possibilités. Il est impératif de mettre en place une solution de **sauvegarde automatique fiable**, qui offrirait la possibilité de **restaurer rapidement** les données. En cas d'incidents, le fait de pouvoir restaurer la donnée perdue minimiserait les temps d'arrêt et préservant la continuité des opérations. Avec une stratégie de sauvegarde bien conçue, les données seront protégées en cas de cyberattaques ou de défaillances matérielles, offrant à l'entreprise une meilleure résilience face aux problèmes potentiels. L'entreprise n'est pas encore au point de définir un **plan de continuité d'activité et Haute disponibilité** ainsi qu'un **Plan de secours informatique** ou **plan de reprise d'activité**, mais cela arrivera sûrement sous peu.

Solution

La solution retenue pour pallier ce problème a été de gérer les sauvegardes automatiques via un programme en **Python**. Ce programme serait responsable de sauvegarder **quotidiennement, hebdomadairement** ou **mensuellement** certaines données en fonction de la **configuration** qui lui sera attribuée. Cette configuration est définie par l'équipe SIG de MODAAL, en attribuant une **importance** à certaines données plutôt qu'à d'autres en fonction de leur volume ou de leur fraîcheur. Il peut être **acceptable de perdre** certaines données si elles sont faciles à réintégrer. Sachant qu'à terme la solution de sauvegarde quotidienne ne sera plus suffisamment efficace et devra impliquer une sauvegarde à l'heure pour certaine donnée.



Les bases de données étant présentes sur **différent serveur de base de données**, il était important de définir pour chaque serveur, quelle base de données, quel schéma ou quelle table doivent être sauvegardés. Cette **configuration** permet de diminuer la **quantité d'espace** que pourrait entraîner la sauvegarde complète de toutes les bases de données de l'entreprise. Chaque information présente dans les bases de données possède son **importance** concernant la capacité de la sauvegarder régulièrement ou non. Par exemple, certaines données libres de droit n'ont pas de nécessité d'être sauvegardées, étant disponibles sûr d'autre plateforme. Une donnée libre de droit mais ayant été retravaillée mensuellement n'a pas la nécessité d'être sauvegardée toutes les semaines.



Pour mettre en place ce principe de **sauvegarde datée**, il est essentiel de conserver la dernière date de sauvegarde pour chaque partie de la configuration. Étant donné que le script s'exécute une fois par jour, pendant la nuit, pour ne pas perturber l'accès aux données par les différents collaborateurs de l'entreprise, il procède à la vérification de la dernière mise à jour pour chaque catégorie (quotidiennement, hebdomadairement, etc.) de chaque base de données. Si une mise à jour est nécessaire, le script lance le processus de sauvegarde de la donnée concernée.

La gestion des risques a permis de définir une méthode de sauvegarde à horaire régulier de **manière planifiée** et **contrôlée**, permettant de **minimiser le risque de perte** de données importantes. Le fait de dater les sauvegardes permet au script de n'exécuter que les sauvegardes nécessaires, ce qui **économise du temps** et des **ressources**.



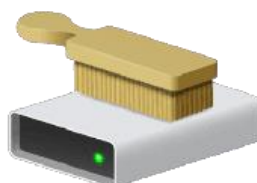
En choisissant de faire tourner le script pendant la nuit, l'entreprise peut s'assurer que les opérations quotidiennes ne sont pas perturbées par le processus de sauvegarde. Cela garantit également que les **données disponibles** pour les différents pôles de l'entreprise sont toujours à jour et fiables.



Pour réaliser ces sauvegardes, **PostgreSQL** propose un outil de sauvegarde directement intégré, appelé **pg_dump**. Avec les paramètres appropriés, **pg_dump** permet de **sauvegarder à chaud** de manière précise n'importe quelle partie de la base de données, sans avoir à inclure l'intégralité de la base. Grâce à cela, il est possible de sauvegarder des **tables spécifiques** ou des **schémas particuliers**, ce qui est très utile dans les environnements avec lesquels seules certaines données nécessitent une sauvegarde fréquente ou une restauration rapide en cas de besoin. La définition de la **fréquence** de sauvegarde est attribuée en fonction de la **quantité de donnée acceptable de perdre** et de la fréquence de mise à jour de celle-ci.

Un **problème** reste à prendre en compte pour définir les options de sauvegarde, la **consommation d'espace disque** qui en découlera. Il est donc crucial de réduire la quantité de données sauvegardées au strict nécessaire. Il reste toujours possible d'augmenter la **taille des disques de stockage**, mais cela entraînerait des **frais supplémentaires** qui pourraient être évités si la configuration des sauvegardes est adaptée aux besoins réels.

Une **solution** à ce problème pourrait être de définir une **hiérarchie des données** en fonction de leur importance et leur fréquence de sauvegarde, en valorisant les **données essentielles** au fonctionnement des différents pôles, ce qui permettrait d'**économiser de l'espace mémoire** et **réduire les coûts** associés au stockage, tout en améliorant l'efficacité du processus de sauvegarde. Cette stratégie pourra se faire appliquer quand le besoin se fera ressentir.



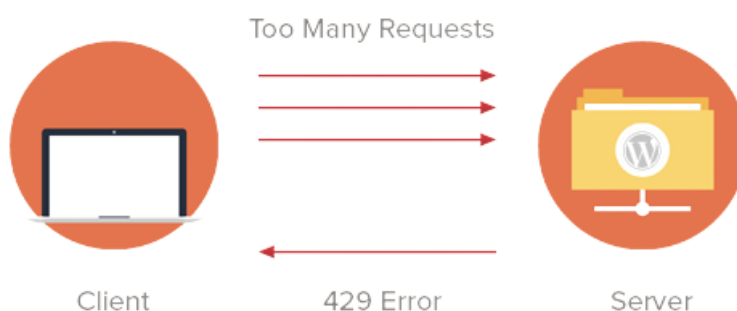


Automatisation du chargement des données

Comme énoncé précédemment, **MODAAL** administre un volume important de **données ouvertes** provenant de services gouvernementaux ou privés. Ces données nécessitent fréquemment d'être retraitées afin de vérifier les informations qu'elles contiennent, d'où l'utilité de l'**API BAN** expliquée précédemment. Cependant, le chargement de ces données devenait fastidieux et nécessitait une **optimisation**. C'est à ce moment que la puissance de **Python** intervient une nouvelle fois, car il est capable de gérer différents formats de données. Il peut récupérer les données ouvertes, les traiter et les importer sur nos serveurs de base de données.

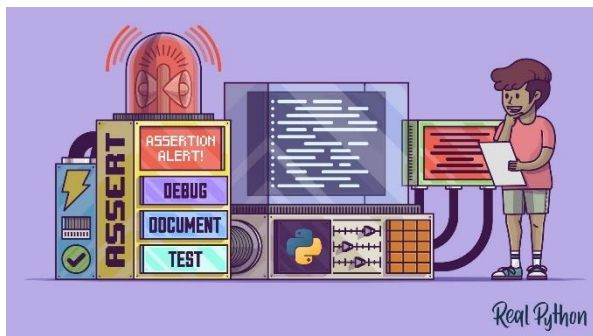
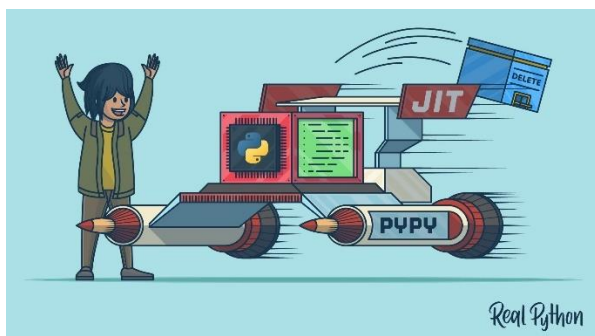
Grâce à Python, le processus de **retraitement des données** devient plus **efficace** et **automatisé**. Il permet de manipuler en grand ensemble de données, ce qui permet de les préparer plus facilement avant de les insérer dans la base de données. Il permet également d'intégrer d'autres fonctionnalités telles que la **validation des données**, la **transformation des formats** ou la **normalisation**, ce qui assure la qualité des données avant leur stockage dans la base de données.

Je me suis donc retrouvé avec la tâche de développer ces scripts de chargement automatique sous la tutelle de **Fabien MARTY**. L'une des principales difficultés, qu'il était important d'essayer de gérer au mieux durant le développement, concerne l'**optimisation du code**. Il fallait faire en sorte que le traitement prenne le moins de temps possible pour récupérer les données depuis le service externe, tout en évitant de se faire **bloquer** à cause du trop grand nombre de requêtes envoyées, puisque la plupart des apis disposent de système de sécurité empêchant l'envoi d'un trop grand nombre de requêtes, pour éviter de surcharger leur serveur.



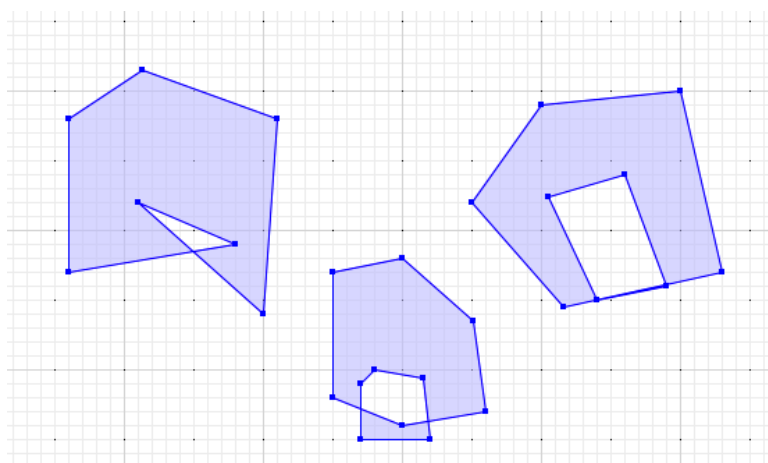
Une fois les données récupérées, il était nécessaire de parcourir celles-ci pour effectuer diverses modifications et **opérations de nettoyage**. La question de l'endroit où effectuer le nettoyage était également importante, car il y avait **deux approches** possibles : envoyer directement les données dans la base de données, puis les nettoyer via des **requêtes SQL** directement dans la base, ou bien les nettoyer **avant de les envoyer**. À la suite de nos tests, nous avons conclu que le nettoyage via Python était plus **optimisé**.

Python offre des **performances élevées**, ce qui réduit considérablement le temps de traitement des données. Il est à la fois **rapide** et **efficace** dans le traitement des données. De plus, sa **flexibilité** permet d'adapter les opérations de nettoyage à chaque cas, ce qui serait plus complexe à réaliser en SQL. En utilisant Python, nous pouvons **gérer les erreurs** plus facilement, car les erreurs survenant pendant le développement sont plus facilement identifiables, **accélérant** ainsi **le processus** de développement.



Cependant, il est important de souligner que notre conclusion est basée sur les besoins spécifiques de notre cas. **Ce n'est en aucun cas une vérité absolue**. En fonction de la taille des données à traiter et de la complexité des opérations de nettoyage, il peut être plus optimisé d'utiliser directement les requêtes SQL.

Par exemple, lorsque nous avons besoin de nettoyer des **géométries** pour vérifier leur validité, modifier leur projection ou réaliser des calculs, l'extension **PostGIS** de **PostgreSQL** est bien plus **adapté**. PostGIS est une **extension géospatiale** puissante qui offre des fonctionnalités spécifiques aux données géographiques. Elle est développée depuis une vingtaine d'années et elle est très populaire ce qui la rend plus adaptée à ce type de traitement.



Géométrie non valide

Liste non exhaustive des données pouvant être chargées automatiquement :

SIRENE :

Le **Système Informatique pour le Répertoire des Entreprises et des Établissements** (SIRENE) est un registre qui répertorie les entreprises et les établissements en France. Il permet de récupérer le numéro SIRET, le statut juridique, l'adresse, l'activité et bien d'autres.



Cadastre :



Le cadastre est un registre officiel des **propriétés foncières**, il donne accès à la superficie, aux limites et aux propriétaires de chaque parcelle de terrain. Les données du cadastre comprennent les adresses et les informations géospatiales des propriétés.

Fichiers des locaux et des parcelles des personnes morales :

Ces fichiers contiennent des informations sur les locaux et les parcelles de terrain détenues par des personnes morales telles que les entreprises, les associations, etc.



Découpages administratifs :

Correspond aux divisions territoriales administratives de la France, les régions, les départements, les arrondissements, les communes, etc.

Géoportail de l'urbanisme et zonage d'urbanisme :

Le Géoportail de l'urbanisme est une plateforme numérique qui fournit des informations géographiques sur les règles d'urbanisme. Le zonage d'urbanisme indique comment les parcelles peuvent être exploitées (résidentiel, commercial, industriel, agricole, naturel, ...).



Infogreffe :



Infogreffe donne accès aux informations légales et officielles des entreprises immatriculées au Registre du Commerce et des Sociétés (RCS), telles que leur statut juridique, leur capital social, leurs dirigeants, etc.

Automatisation de tâches

Webinaire Zoom

Le mot « **webinaire** » est un mot-valise né de la combinaison des mots « **web** » et « **séminaire** », et il désigne une **conférence en ligne** où des participants provenant de différents endroits peuvent se connecter. Les personnes voulant participer doivent s'inscrire pour pouvoir accéder à la conférence. Pour cela, ils utilisent un **lien** qui leur permettent de rejoindre le logiciel sur lequel se déroulera la conférence. Cette plateforme offre des possibilités d'échanges en **visioconférence** et **audio**. Tout en proposant d'**enregistrer les séances**, pour être **revisionnées**, ou **partagées** aux personnes ne pouvant participer au webinaire. Ils ont aussi la possibilité de partager des documents, de projeter leur écran d'ordinateur, d'envoyer des messages depuis une messagerie globale et collaborer sur un même espace d'édition.

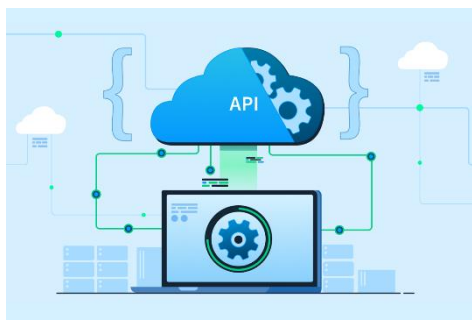


Exemple de Webinaire proposé par MODAAL

Les webinaires organisés par **MODAAL** se sont principalement déroulés sur **Zoom**, un logiciel de visiophonie développé par la société Zoom Video Communications. Zoom possède tous les outils nécessaires au bon déroulement du webinaire, tel que la possibilité d'enregistrer ses sessions. En 2019, plus de la **moitié** des entreprises du classement **Fortune 500** auraient utilisé Zoom, ce qui est un très bon indice sur la popularité de cet outil.



Dans le cadre d'un suivi complet du déroulement du webinaire, il était essentiel de recueillir de nombreuses informations pour établir des **statistiques** pertinentes. Cela



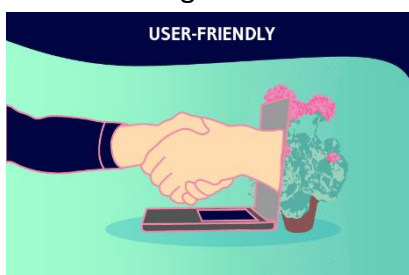
impliquait l'utilisation de **l'API de Zoom**, qui permet de récupérer automatiquement des informations, rendant le processus bien plus **efficace** que de le faire **manuellement** depuis l'interface du webinaire. Dans notre cas, l'accent était principalement mis sur la récupération de la liste des **personnes inscrites** avec toutes les informations disponibles les concernant.

Pour interroger l'API de Zoom et récupérer les informations des webinaires qui nous intéressaient, nous avons utilisé un



package Python spécifique pour la gestion des **authentifications**, appelé « **JWT** » (JSON Web Token). Ce package nous a permis de générer **un jeton d'authentification crypté** en utilisant la clé d'API récupérée depuis le tableau de bord de Zoom. Ce jeton nous a donné **l'autorisation** nécessaire pour nous identifier auprès de l'API de Zoom de manière **sécurisée**, ce qui nous a ensuite permis de récupérer les données pertinentes concernant les webinaires.

Le script devait être exécuté régulièrement et par plusieurs collaborateurs, ainsi il devenait obligatoire de faciliter l'utilisation de ce script pour les utilisateurs, de le rendre



« **user-friendly** ». Le terme anglais « user-friendly », décrit la nécessité de créer une interface **intuitive** et **facile à comprendre**, permettant aux utilisateurs d'interagir avec le script sans complications inutiles. Il était nécessaire de créer une interface utilisateur **facile d'utilisation** rendant son utilisation simple par tous les utilisateurs, quel que soit leur **niveau de compétence technique**.

La première chose à faire était de **minimiser les interactions** que l'utilisateur devra effectuer, la seule information qui était obligatoire pour récupérer la liste des participants était **l'identifiant du webinaire**. Le script se devait d'être installé par plusieurs collaborateurs, c'est-à-dire qu'il puisse être lancé depuis n'importe quelle machine sans installation préalable. Il existe un package python nommé « **PyInstaller** », un outil très pratique permettant de créer des **exécutables** à partir des fichiers bruts. Ainsi, j'ai pu générer un fichier exécutable du script en question, facilitant son utilisation sans nécessiter l'installation de Python ou d'autres dépendances spécifiques. Il suffisait de rentrer l'identifiant du webinaire et l'exécutable s'occupait de générer **un fichier CSV** contenant ces informations, ce qui a rendu l'exécution du script accessible à un plus **large public** sans souci de configuration complexe.



Scraping de données



Le **web scraping**, ou moissonnage en français, est le principe de **collecter** des **données** depuis des pages **web**, ce qui peut parfois être compliqué, fastidieux, voire impossible, avec les méthodes conventionnelles telles que le téléchargement ou le copier-coller. Le développement d'un logiciel ou d'un script peut rendre l'**extraction automatique** des données bien plus simple. Le web scraping permet de récupérer le contenu d'une page web de manière **structurée** et permet la réutilisation de ces données pour diverses applications bien plus simple.

Dans le contexte de MODAAL, le web scraping avait pour but de **collecter des contacts** disponibles dans les listes du site de la [Fédération des Promoteurs Immobiliers](#) ainsi que du site [CFNewsImmo](#) (Corporate Finance News Immobilier). Cette démarche visait à enrichir la liste de prospection pour la commercialisation des outils que MODAAL propose.

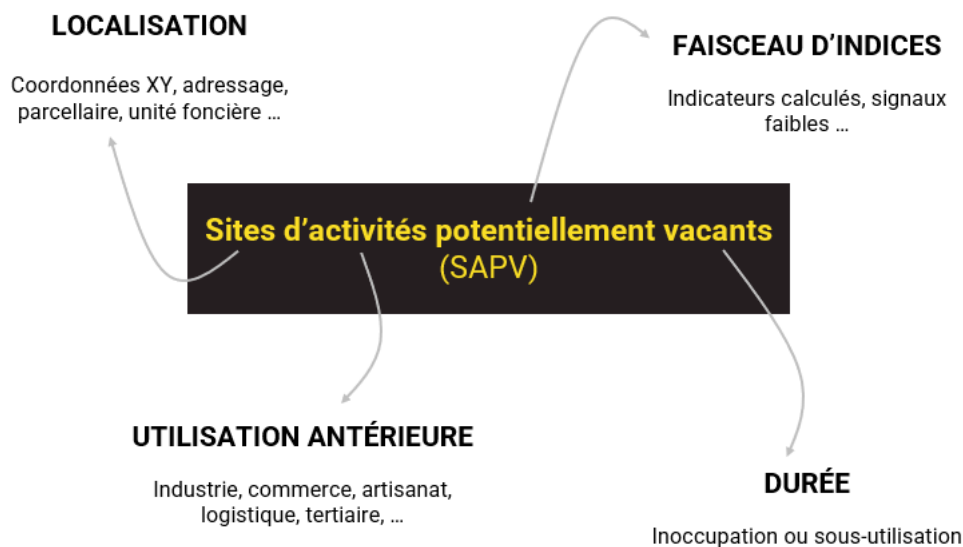


Ayant déjà pratiqué le scraping au cours de mes années d'apprentissage en informatique, j'ai naturellement proposé un outil qui m'est **familier** et qui semblait être le plus **optimisé** pour réaliser cette mission puisqu'il a la capacité de gérer le **JavaScript** que peut contenir une page web. Cet outil est une bibliothèque **Python** nommé **Selenium**. Selenium est un **framework open-source**, originalement développé dans le but d'exécuter des **tests automatisés** sur la plupart des **navigateurs** tels que Chrome, Internet Explorer et bien d'autres. Cette bibliothèque est pour autant largement capable de réaliser du web scraping. De cette manière, le logiciel devient capable de naviguer entre les pages internet et d'interagir avec les éléments de la page (**DOM**) comme s'il était un utilisateur réel. Ainsi, là il devient bien plus simple de scraper des sites web dynamiques, c'est-à-dire des sites qui affichent des résultats en fonction des actions effectuées par l'utilisateur. Ainsi, Selenium s'avère être un outil précieux pour collecter des données à partir de sites web interactifs et complexes.

En créant un **bot** grâce à un script Python, j'ai offert à **Guillaume HAMEAU**, apprenti Marketing Digital chez MODAAL, sous la tutelle d'**Edouard MOUGIN**, un **gain de temps** considérable. Il a pu récupérer la liste de tous les contacts sans avoir à effectuer des opérations fastidieuses de copier-coller pour chaque profil de chaque site.

Un bot informatique est un **logiciel automatisé ou semi-automatisé** qui agit comme un agent en interagissant avec des serveurs informatiques. Il se connecte et communique avec le serveur de la même manière qu'un programme client utilisé par un humain. C'est pourquoi le terme « bot » est une abréviation du mot « **robot** » par [aphérèse](#).

Sites d'activités potentiellement vacants



Introduction

MODAAL intervient de manière régulière auprès d'**acteurs publics** ou **privés** dans toute la France pour aider à la gestion foncière et le **recensement des friches**. Ces interventions ont permis à MODAAL d'acquérir une connaissance approfondie des différentes méthodes existantes utilisées pour **identifier les friches** et des **bases de données** mobilisables pour cette tâche. Grâce à cette expérience terrain, MODAAL a pu affiner son approche et développer une méthode **efficace** pour **détecter** les sites vacants.



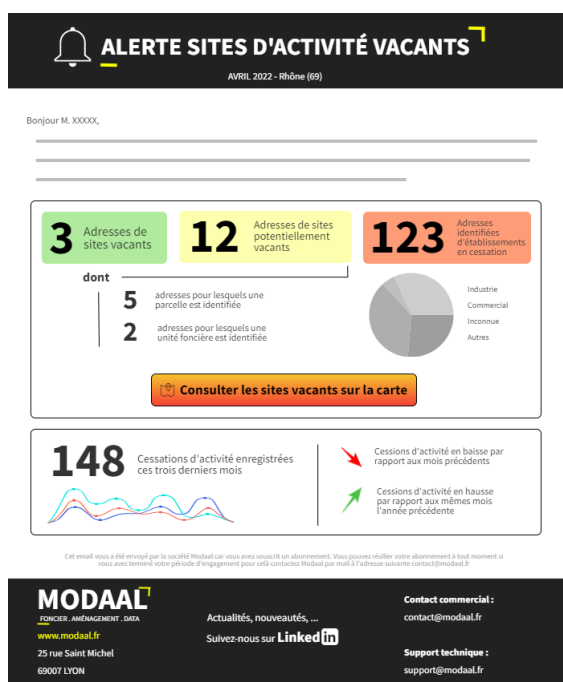
MODAAL propose une **approche novatrice** et large pour identifier les friches, englobant une notion plus vaste que celle traditionnellement utilisée, permettant d'obtenir un spectre plus large dans le résultat des sites. Selon la **loi Climat et Résilience**, une friche est définie comme étant « *tout bien ou droit immobilier, bâti ou non bâti, inutilisé et dont l'état, la configuration ou l'occupation totale ou partielle ne permet pas un réemploi sans un aménagement ou des travaux préalables* » (Art. L111-26 Code de l'Urbanisme). Cependant, l'état technique ou de dégradation, qui permettrait de juger si une friche est **réhabilitable sans aménagement** ou travaux préalables, n'est pas pris en compte faute de base de données fiable disponible, impliquant l'élimination de ces potentiels friches des radars. En conséquence, les sites identifiés par MODAAL répondent à une définition plus large, incluant des biens dont l'état technique pourrait nécessiter des aménagements ou travaux préalables.

La proposition est de rendre cette **capacité d'identification** de sites disponible à **un large public**, y compris au secteur privé. Il n'existe pas de solution miracle ni de jeu de données exhaustif pour identifier toutes les friches d'un territoire, mais l'approche développée par MODAAL permet de trouver des **sites pertinents** et **vacants**. Cette approche a été mise à l'épreuve par des comparaisons sur des territoires tests à l'échelle départementale, et les résultats ont démontré son efficacité.

Mes réalisations

Méthode de détection des sites

J'ai contribué à la mise en place de certains scripts dédiés à la **détection des sites potentiellement vacants** et à l'installation de l'**API BAN** sur nos serveurs qui s'est avérée extrêmement utile pour le bon fonctionnement de ces scripts. Cependant, l'une des responsabilités qui m'a été confiée et sur laquelle j'ai joué un rôle plus important était le développement de la fonction d'**envoi automatique d'emails** contenant les informations relatives aux nouveaux sites identifiés, à destination des utilisateurs inscrits, ce qui permet de maintenir les utilisateurs informés de façon régulière des dernières opportunités détectées.



Maquette initiale de l'email

Les informations contenues dans l'email ne sont pas sensibles, mais ne sont accessibles que à la suite de la souscription d'un abonnement. Il était quand même nécessaire de **garantir la sécurité** lors de l'envoi des emails, pour ce faire, j'ai implémenté le **protocole SSL/TLS** dans le code. Le protocole **SSL** (Secure Sockets Layer) permet de définir un **certificat d'authentification** et **TLS** (Transport Layer Security) reprend les principes utilisés par le protocole SSL tout en ajoutant une **clé de cryptage pour les échanges de données**. Grâce au protocole SSL, j'ai pu créer une connexion sécurisée avec le serveur de messagerie, il m'a permis de générer un certificat avec le serveur permettant de s'identifier pour être certain que personne n'intercepte notre mail. En utilisant les informations de configuration, j'ai ensuite instancié un objet **SMTP** (Simple Mail Transfer Protocol) pour spécifier le serveur SMTP et le port de connexion.



La structure de l'email demandée par le chef de projet devait inclure des **représentations graphiques** permettant ainsi de rendre la lecture de l'email plus simple pour le client. Il existe des bibliothèques nommées **matplotlib** et **numpy** qui m'ont aidé pour créer des représentations graphiques propres. Matplotlib est une bibliothèque de **visualisation de données**, elle permet de créer des **diagrammes** de types variés tout en les rendant totalement **personnalisables**. Numpy, est une bibliothèque qui permet de gérer efficacement des tableaux et des opérations mathématiques. J'ai pu facilement manipuler les données extraites des sites détectés et les représenter graphiquement sous forme de diagrammes, rendant ainsi les informations plus **compréhensibles** et **accessibles** aux utilisateurs.



La structure d'un email de base, « **plein texte** », ne permet pas l'affichage d'**image**, il était nécessaire d'utiliser le **format d'email « HTML »**. Il permet d'exploiter les possibilités de codage HTML, ce format permet une mise en forme **visuelle améliorée** et met en valeur le contenu textuel, tout en permettant l'incorporation d'images pour renforcer l'**impact** du message. Malheureusement, la création d'un email en HTML présente quelques **inconvénients** qu'il est essentiel de prendre en compte pour éviter d'avoir un email **mal mis en page** et donc **illisible** pour le client.

Parmi les inconvénients du format HTML, on peut mentionner la **complexité** de la création. Pour que le code HTML puisse être **interprété** par toutes les messageries, il faut adapter le code, ce qui rend le développement d'un email **responsive** complexe et implique la nécessité de tester et de valider la compatibilité sur différents clients de messagerie pour assurer une présentation cohérente sur tous les appareils.

border-radius

The `border-radius` CSS property rounds the corners of an element's outer border edge.

Estimated Support /Read
73.17% + 14.63% = 87.8%

Apple Mail		Gmail				Outlook						Yahoo! Mail			AOL		
macOS	iOS	Desktop Webmail	iOS	Android	Mobile Webmail	Windows	Windows Mail	macOS	Outlook.com	iOS	Android	Desktop Webmail	iOS	Android	Desktop Webmail	iOS	Android
						2003											
						2007											
						2010											
						2013											
						2016											
10.3	12.2	2019-08	2019-08	2019-08	2020-02	2019	2020-01	2016	2019-08	2019-08	2019-08	2021-02	2021-03	6.18.2.1	2021-02	2021-03	2021-03

Exemple de propriété css incompatible

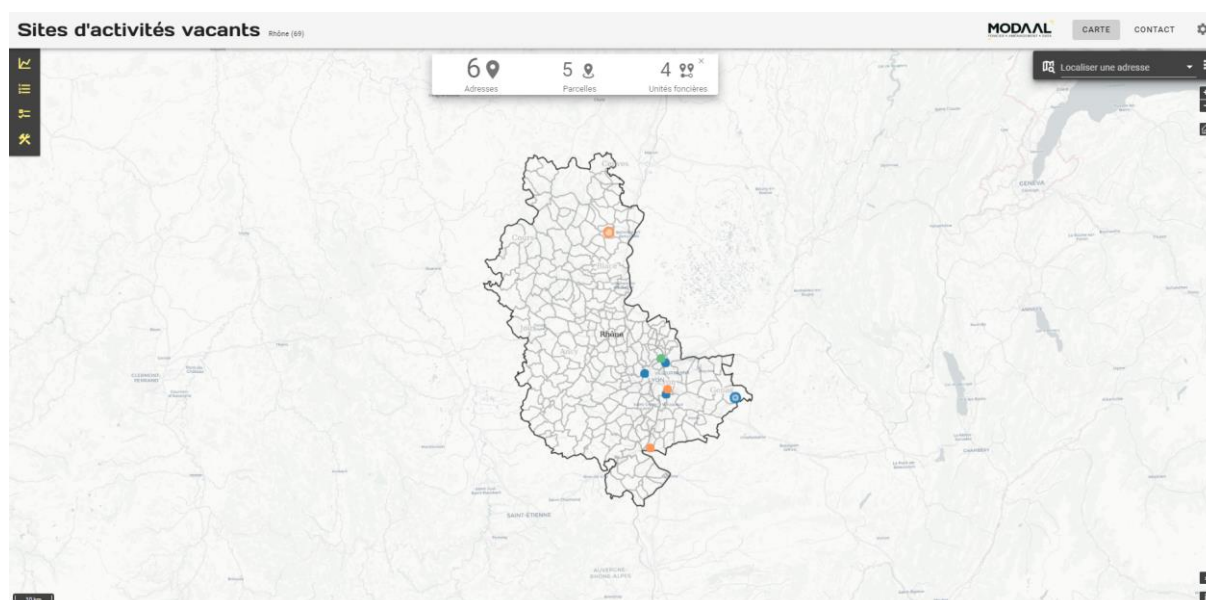
De plus, certains **filtres anti-spam** peuvent être plus stricts avec les emails en HTML, ce qui peut affecter la livraison du message. Le résultat de l'email envoyé aux clients est disponible en [annexe](#).

Tableau de bord utilisateur

La solution mise en place a offert un **accès pratique aux informations** sur les friches. Chaque utilisateur reçoit des notifications concernant les nouvelles friches détectées, accompagnées d'une **URL personnalisée** envoyée par email. Cette URL contient un **jeton d'authentification** et le département spécifié, assurant ainsi la sécurité et l'authentification de l'utilisateur avant d'accéder à la carte. En cas d'accès à un département non autorisé, l'utilisateur est **redirigé** vers une page de contact. Grâce à cette interface intuitive, l'utilisateur peut facilement **visualiser** la carte zoomée sur le département choisi et explorer les sites vacants détectés au cours du dernier mois.

J'ai été chargé du développement du tableau de bord utilisateur, conçu pour reprendre les informations contenues dans l'email de manière **géographique**, facilitant ainsi la lecture pour l'utilisateur. Des informations complémentaires sur chaque site détecté sont également fournies. Pour le développement du tableau de bord, j'ai choisi de rester fidèle aux technologies utilisées dans **GéoFriches®**. En effet, le composant principal demeure la carte, qui était déjà développée et offrait une expérience utilisateur complète. Ainsi, l'application reprend les fonctionnalités essentielles de GéoFriches®.

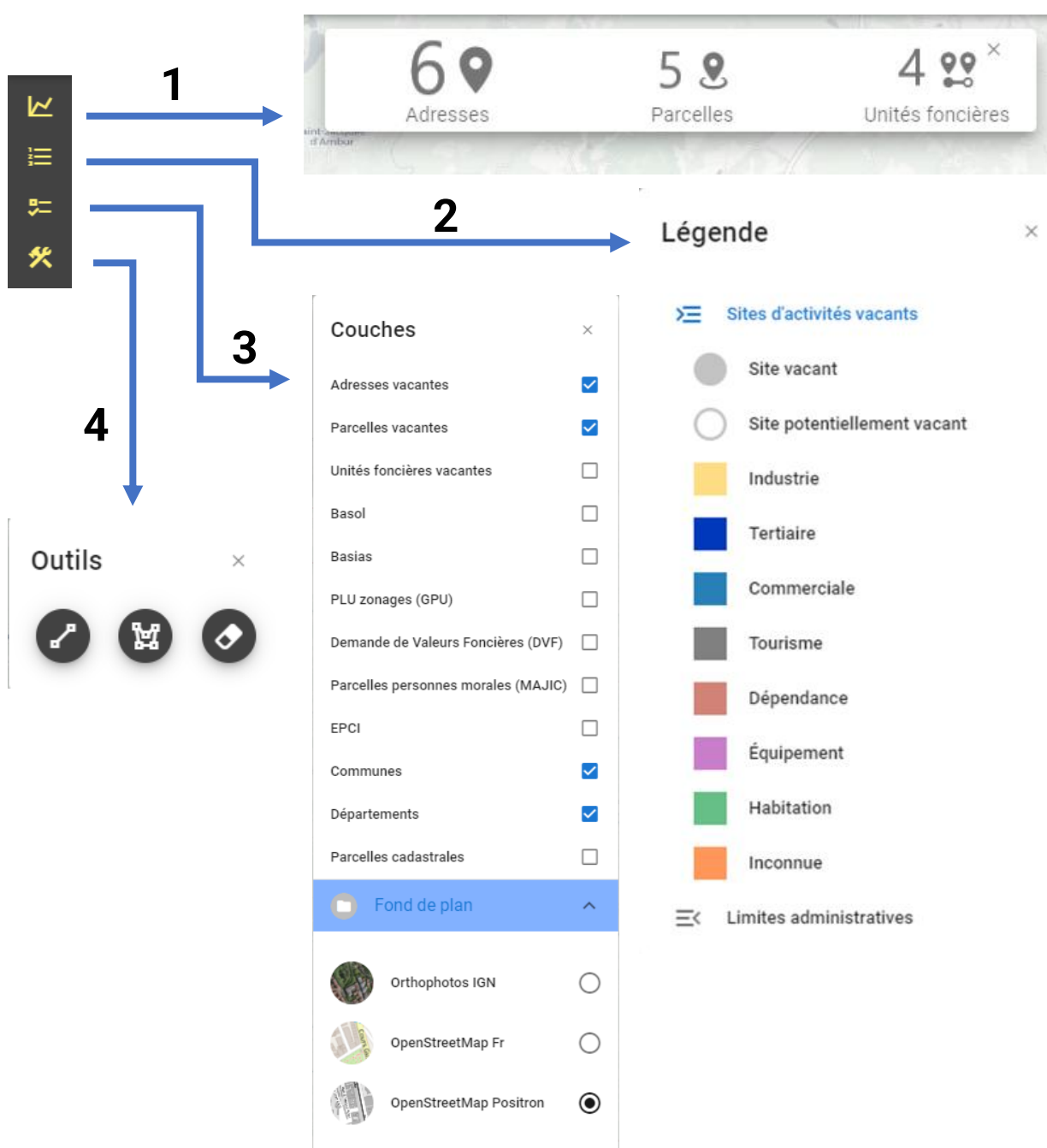
Les utilisateurs peuvent consulter la carte contenant les sites vacants du dernier mois pour le département spécifié. Les fonctionnalités de base de la carte sont intégrées, offrant un **affichage clair** des indicateurs, de la légende et du gestionnaire de couches. De plus, les outils de mesure permettent à l'utilisateur d'obtenir des informations supplémentaires sur les friches, tandis que les options d'impression facilitent la génération de rapports personnalisés. La localisation par adresse et la sélection par clic droit ou gauche offrent des fonctionnalités avancées pour une analyse approfondie des sites vacants.



Carte affichée via le lien de l'email

Menu disponible

La carte est équipée d'un outil de localisation qui utilise l'**API BAN** (Base Adresse Nationale) pour effectuer des **recherches d'adresses**. Les utilisateurs peuvent facilement trouver des sites spécifiques en saisissant une adresse, et la carte se déplacera automatiquement vers l'emplacement recherché. Pour rendre la carte le plus **interactive** possible, nous utilisons une bibliothèque cartographique qui fournit énormément d'outils et de méthodes, c'est **OpenLayers**. Ces fonctionnalités avancées assurent une expérience utilisateur optimale en offrant des **outils intuitifs** et **puissants** pour explorer les friches et les informations associées.



1 : L'utilisateur peut afficher dans une fenêtre, **les informations reçues** par email directement sur la carte interactive.

2 : La fenêtre propose une **légende dynamique** qui s'ajuste automatiquement en fonction des choix de l'utilisateur concernant les informations à afficher. Dans l'exemple fournit plus haut, on peut remarquer les destinations des sites détectés, qui sont représentées par des couleurs différentes. La destination de ces sites a été défini lors de la catégorisation (qualification) des données générées par le script ou par les collaborateurs de MODAAL.

3 : Un **gestionnaire de couches** pour sélectionner les couches affichées sur la carte, adaptées aux besoins des utilisateurs ce qui offre une expérience personnalisée en lui permettant de visualiser uniquement les données qui l'intéressent.

4 : Des **outils de mesure** sont également disponibles, permettant à l'utilisateur de calculer une superficie ou une distance directement en dessinant sur la carte.



L'utilisateur peut aussi accéder à **ses données personnelles**, depuis son profil dans les paramètres. Il peut les **consulter**, mais aussi les **modifier** directement depuis le tableau de bord. Cette fonctionnalité lui offre un contrôle total sur ses informations.

De plus, pour faciliter la **communication** avec les équipes de MODAAL, un formulaire de contact est mis à disposition ce qui permettra à l'utilisateur de faire parvenir un problème rencontré ou de partager ses remarques concernant l'application ou les données affichées sur la carte.

En [annexe](#), vous trouverez un aperçu des pages disponibles, pour vous permettre de visualiser l'interface utilisateur.

Authentification utilisateur

Sites d'activités vacants Rhône (69)

Un utilisateur ne peut voir que les informations concernant le département qui est compris pour l'**abonnement mensuel** souscrit. Il peut bien sûr voir plusieurs départements, mais il recevra un email différent pour chaque département puisque les scripts de détection fonctionnent avec une **zone géographique départementale**.

Concernant l'accès à son espace client, il aurait été possible d'ajouter une **page de connexion** vers le tableau de bord pour **identifier** l'utilisateur et ainsi s'assurer qu'on n'affiche pas les informations d'un autre département. Cependant, cela aurait pénalisé l'expérience utilisateur en ajoutant **des étapes supplémentaires** et nuisible à la fluidité de l'outil. Pour contourner ce problème, j'ai proposé une solution alternative consistant à générer des **URL distinctes** pour chaque utilisateur, **un système d'authentification forte par jeton**.

Ces URLs sont régénérées à chaque envoi d'email et contiennent le **jeton d'authentification** (token) de l'utilisateur ainsi que l'identifiant du département à afficher. En cliquant sur l'URL personnalisée, l'utilisateur peut accéder directement à son tableau de bord sans avoir à se connecter. Cette approche **simplifie l'expérience utilisateur** en évitant les étapes de connexion, et elle **garantit la sécurité** grâce à la complexité du jeton généré aléatoirement.

Pour renforcer la sécurité, les URLs générées doivent être suffisamment **complexes** et **aléatoires** pour empêcher toute tentative d'accès non autorisé. De plus, elles sont **cryptées** pour empêcher leur manipulation. Cette méthode fonctionne bien dans notre cas, puisque les données affichées dans le tableau de bord ne sont ni sensibles ni confidentielles, ce qui réduit le risque d'attaque ciblée.

Lorsque l'utilisateur clique sur le lien contenu dans l'email, le back-end de l'application s'occupe de vérifier que le **token** fournit dans l'url du lien cliqué soit toujours **valide** pour accepter l'envoi des informations associées. Si tout est conforme, les informations du tableau de bord sont alors affichées. En cas d'accès à une URL **non valide ou non autorisée**, l'utilisateur est **redirigé** vers la page de contact non connecté pour obtenir de l'aide.

En complément, j'aimerais vous parler de **missions annexes**, qui sont directement liées à ma problématique sur le développement de compétences en informatique.

Je me suis retrouvé en charge de gérer la gestion **du passage de Vue 2 à Vue 3**. Le suivi et la mise à jour des technologies utilisées est une étape importante pour rester **compétitifs** et **efficaces**. Le passage de Vue 2 à Vue 3 a été une phase importante dans le suivi des **actualités technologiques**. Développer des compétences en informatique permet de **maintenir** et de **moderniser** les applications existantes. Le passage par une optimisation avec de **l'expérience** permet **d'améliorer les performances, la stabilité, la sécurité** et d'assurer leur compatibilité avec les nouvelles technologies.

Je suis également intervenu dernièrement, en tant que **soutien**, pour la réalisation d'un projet de tableau de bord pour visualiser l'avancement des étapes des traitements d'identification des sites vacants réalisé par une stagiaire, **Claire BOUCHARD**. La **collaboration** et le **partage** de compétences sont des aspects clés de la réussite d'un projet. En intervenant et en suivant le projet de tableau de bord, j'ai pu démontrer l'importance de la **formation** et du **transfert de compétences**. Cette initiative a permis d'initier la stagiaire au développement informatique sur un projet simple de suivi d'exécution de traitements.

En recherche et développement, j'ai essayé d'automatiser **la classification d'articles** grâce à des modèles de [deep learning](#) pour vérifier l'intérêt des articles trouvés : Cette tâche, toujours en cours, sera essentielle pour optimiser le processus de **veille informationnelle** du bureau d'études. En utilisant des modèles de deep learning, j'ai pu commencer à **automatiser la détection d'articles intéressants** par rapport aux thématiques pertinentes pour le bureau d'études, ce qui permettra de gagner du temps dans la collecte de données et d'identifier rapidement du contenu pertinent pour les projets en cours.

Le développement de compétences en informatique permet à un bureau d'études en urbanisme **d'ouvrir le champ des possibilités** en réduisant ces limitations, grâce à des méthodes de travail et des outils devenant de plus en plus **performant**. Les connaissances en programmation et en gestion de projets ont donné la possibilité **d'automatiser des tâches récurrentes** ou fastidieuses, **d'améliorer la rapidité** de certains traitements et de proposer des outils répondant aux besoins du bureau d'études. Ces expériences m'auront permis de découvrir un domaine d'activité qui au premier abord ne me semblait pas du tout correspondre au métier de développeur. Aujourd'hui j'ai complètement changé d'avis, et remarqué que **les compétences en informatique est un facteur déterminant** pour le bureau d'études dans son évolution et sa compétitivité sur le long terme.